DIFFERENT DATABASE TYPES

- Companies require databases to store, maintain and update a wide range of data from employee information to customer data.
- Each database has different capabilities and storage capacity depending on how much data a company needs to store, how many users they have and how complex their data is.
- Knowing the different database types can help one determine which might be best for a business.
- The two primary types of databases are:
- **Single-file:** single-file, or flat-file databases, use simple structures and individual files to represent one piece of data or information.
- **Multi-file relational:** relational databases are more complex databases that use tables to show the relationship between data.

- In the digital landscape, the backbone of any robust application or system resides in its database architecture.
- Choosing the appropriate database, considering the Relational vs. Non-Relational Databases debate, is pivotal for businesses in determining performance, scalability, and the ability to handle diverse data categories.
- However, choosing the right database can be really tricky.
- There are so many options out there, and it’s tough to figure out which one fits the needs.

Relational vs. non-relational

- There are essentially two major types of databases, NoSQL and Relational, with all the others being different versions of these.
- A NoSQL database has a hierarchy similar to a file folder system and the data within it is unstructured.
- This lack of structure allows them to process larger amounts of data at speed and makes it easier to expand in the future.
- Cloud computing regularly makes use of NoSQL databases.

Relational databases

- Relational databases organize data into tables with rows and columns, guided by predefined schemas and enforced relationships.
- They use SQL for standardized data manipulation and ensure robustness through ACID compliance, making them ideal for structured data with complex relationships and transactional integrity.

Non-relational databases (NoSQL)

- NoSQL databases, are diverse in models like document-based, key-value, columnar, and graph structures, catering to unstructured or semi-structured data.
- They offer scalability, flexibility, and varied consistency models, allowing trade-offs between performance and consistency.
- NoSQL databases excel in scenarios requiring high-speed processing, scalability, and managing dynamic data structures.

Choosing between them

- Relational databases suit structured data with strict schemas and complex relationships, ensuring data integrity in transactions.
- In contrast, non-relational databases excel in managing unstructured or rapidly changing data, offering scalability and flexibility.
- The choice between Relational vs. Non-Relational Databases hinges on aligning each database type's strengths with the specific needs of the application or business, ensuring optimal data management and performance.

Data models and flexibility

- Relational databases organize data into tables with rows and columns governed by fixed schemas defining data types and relationships.
- While ensuring data integrity and consistency, the rigid schema can make adapting to evolving data patterns challenging.

- Non-relational databases, like document-based, key-value stores, columnar, and graph databases, offer diverse structures catering to specific data needs.
- *Document-Based:* schema-less documents enable flexible data structures that quickly adapt to changes without strict schema constraints, ideal for dynamic or unstructured data.
- *Key-Value Stores:* These databases provide simplicity and rapid data retrieval but offer limited query capabilities compared to other models suitable for caching and quick data access.
- *Columnar Databases:* Optimize query performance for analytics and handling large datasets efficiently.
- *Graph Databases:* Designed for complex relationships between data points, they excel in network analysis and recommendation systems.

Scaling

- Non-relational databases shine in horizontal scalability, distributing data across multiple servers seamlessly.
- Models like document-based and key-value stores excel in handling unstructured data, allowing effortless scaling to accommodate growing demands.

Performance

- Relational databases prioritize transactional consistency and excel in managing complex transactions.
- Non-relational databases offer varied performance traits based on their models, optimizing data access and query capabilities for specific needs.
- Relational databases suit scenarios needing strict transactional integrity, while non-relational databases, with their horizontal scaling prowess, are ideal for dynamic applications.
- Balancing scalability and performance involves aligning database capabilities with anticipated growth, ensuring an efficient and adaptable architecture.

Query languages and indexing

- SQL enables complex operations like joins and aggregations across multiple tables, making it robust for structured data handling in systems like PostgreSQL and MySQL.
- Non-relational databases use tailored query languages like MongoDB Query Language (MQL) to navigate their unique structures efficiently, optimizing data retrieval.
- Relational databases offer diverse indexing techniques to enhance query performance.
- Non-relational databases provide specialized indexes (e.g., single field, compound) catering to varied data models.
- Relational databases adhere to ACID (Atomicity, Consistency, Isolation, Durability) principles, ensuring transactional integrity and data consistency even in concurrent operations.
- Non-relational databases often offer flexibility in consistency models, allowing trade-offs between performance and data integrity based on application needs.

Choosing based on needs

- Relational databases suit applications needing strong consistency.
- Non-relational databases offer flexibility, allowing the choice of consistency levels based on specific requirements.

Use cases and applicability

- Relational databases are ideal for structured data with complex relationships:

    Financial Systems

    Inventory Management

    CRM Systems

    E-commerce Platforms

- Non-relational are flexible and scalable for varied data structures:

  Real-Time Analytics

  Content Management Systems

  IoT Environments

  Social Media and Recommendations

Choosing based on applicability

- Relational databases suit structured data with established relationships, ensuring transactional integrity.
- Non-relational databases manage evolving data structures and unstructured data, offering scalability and adaptability.

Community support and ecosystem

- The community and ecosystem surrounding a database are crucial for its development, support, and evolution.
- Relational databases like PostgreSQL and MySQL boast extensive communities with decades of development and refinement.
- They offer:
- *Mature Ecosystems*: Rich with documentation, forums, and resources aiding developers.
- *Vast Toolsets*: A wide array of tools and extensions enhancing database functionality.
- *Active Contributions*: Regular updates and enhancements driven by a committed user base.

- Non-relational databases like MongoDB and Cassandra also thrive on active and evolving communities:
- *Vibrant Development*: Constant evolution to meet modern data challenges and user needs.
- *Growing Communities*: Expanding user base contributing to innovative solutions.
- *Adaptive Tools*: Evolving toolsets and integrations catering to diverse data models.

Support

- Relational databases offer mature ecosystems with established tools and robust support, ensuring stability and reliability.
- Non-relational databases, while newer, boast dynamic communities and tools adapting to modern data demands, providing innovative solutions and rapid evolution.

Note:

- NoSQL has the following disadvantages.

  NoSQL is an open-source database.

  GUI is not available

  Backup is a weak point for some NoSQL databases like MongoDB.

  Large document size.

Other database types

- As database technology has improved over the years, so have the different databases.
- There are many types of databases, each with its strengths and weaknesses based on its design.
- It is especially important for businesses to understand the different databases, to ensure they have the most efficient setup.
- The two primary types of databases are as follows:
- **Single-file:** single-file, or flat-file databases, use simple structures and individual files to represent one piece of data or information.
- **Multi-file relational:** relational databases are more complex databases that use tables to show the relationship between data.

- Apart from the two most primary databases, companies use different databases for the various tasks they perform.
- Depending on the industry and business requirements, a company may use one or many of these:

Centralized database

- A centralized database is one that operates entirely within a single location.
- Bigger organizations like corporations or universities typically use centralized databases.
- The database itself is on a central computer or database system and users can access the database through a computer network, but it is the central computer that runs and maintains the database.
- The advantages are that there is improved data security, reduced redundancy and consistency.
- However, the size of the centralized database is large which increases the response and retrieval time and also It is not easy to modify, delete and update.

Cloud database

- Cloud databases run on cloud computing networks.
- Rather than storing data on local devices or purchasing dedicated storage hardware, companies can install software to access large amounts of data in the cloud.
- Companies often choose cloud databases because they are easy to use.
- They can easily increase storage capacity and they often back up the data on remote servers.
- There are two main types of cloud databases:
- **Traditional:** traditional cloud databases are databases where companies host the database on their servers and the IT team troubleshoots any issues.
- **Database-as-a-Service:** DBaaS is when companies offer their cloud databases to companies with additional services like automation and technical support.

Commercial database

- Commercial databases are databases created by companies and sold to other businesses.
- Businesses typically pay a licensing fee for these.
- The vendor often provides technical support to the businesses rather than internal IT teams or community support.
- These are high-performing databases that are reliable and scalable for growing businesses.

Distributed database

- Distributed databases store data at various physical locations.
- This can mean companies store data on different physical computers at a single location or hardware across different locations on a network.
- Distributed databases use processes called replication and fragmentation when storing data.

- Replication is when the database stores two or more copies of the same file at different locations while fragmentation is when the database divides files into smaller pieces that they then store in different locations.
- Distributed databases are constructed to achieve high availability.

End-user database/ Personal Databases/Single user databases

- End-user is a term used in product development that refers to the person who uses the product.
- An end-user database is, therefore, a database that is primarily used by a single person.
- A good example of this type of database is a spreadsheet stored on the local computer if you own a business.
- It is easy to use and occupies less space.

Graph database

- Graph databases focus on the relationships between data along with the data itself.
- The database uses structures of connected graphs of nodes and edges for users to run queries.
- Companies may choose graph databases if they have datasets with complex relationships.

Hierarchical database

- Hierarchical databases store data in categories that get more specific.
- The databases store data as tree diagrams, showing the relationship between data elements.
- The top-level data is the root or parent, below that is the child or level one and below that is level two.
- For example, if the data is information about a particular technology market, the parent may be a category like computers, the child might be laptops, and the level two could be a specific brand.

Network database

- Network databases allow the data to be linked to over one parent file.
- Visually, companies represent this as an upside-down tree where each piece of data links to the primary data.
- With network databases, users link many data points to other bits of information, creating a web or network.
- Different from the hierarchical model, relationships span multiple categories or trees.

Objected-oriented database

- Object-oriented databases are ones in which the data is represented as objects and classes.
- An object is a real-world item, such as a name or phone number, while a class is a group of objects.
- Objected-oriented databases are a type of relational database.
- Consider using an object-oriented database when there is a large amount of complex data to process quickly.

Open-source database

- An open-source database is a public database that all people can use for free.
- Unlike commercial databases, users can download or sign up for open source databases without paying a fee.
- The term "open source" refers to a program in which users can see how companies make it and they can edit the program to fit their needs.
- Open-source databases are typically much cheaper than commercial databases, but they can also lack some of the more advanced features found in commercial databases.

Operational database

- The purpose of an operational database is to allow users to modify data in real-time.
- Operational databases are critical in business analytics and data warehousing.
- Companies set these up either as relational databases or NoSQL, depending on needs.
- Conventional databases rely on batch processing, where they process commands in groups. Operational databases allow you to add, edit and remove data at any moment, in real-time.

Datawarehouse

- It aggregates large volumes of data from multiple sources into a single repository of highly structured and unified historical data.
- The centralized data in a warehouse is ready for use to support business intelligence (BI), data analysis, artificial intelligence, and machine learning needs to inform decision making and improve organizational performance.

INTRODUCTION TO STRUCTURED QUERY LANGUAGE (SQL)

- SQL is used to manage and manipulate databases.
- Whether one is a beginner or a seasoned developer, understanding the basics of this programming language is essential for anyone working with data.
- SQL is an essential language for building and maintaining relational databases, which opens the door to a range of careers in the data industry and beyond.
- Used in data science, analytics, and engineering, SQL makes it easy to work with data and make more informed strategy, operations, and business decisions.
- It’s a helpful skill for anyone who works with data (even in non-tech roles).

Types of SQL commands

- SQL commands are grouped into several categories like:
- **Data definition language (DDL)** – DDL commands are used to create and modify a database's structure, such as creating tables, altering table structures, and deleting tables. Some examples of DDL commands include CREATE, ALTER, and DROP.
- **Data manipulation language (DML)** – DML commands are used to manipulate the data within a database. These commands include SELECT, INSERT, UPDATE, and DELETE.
- **Data control language (DCL)** – DCL commands are used to manage access such as granting and revoking permissions. Examples of DCL commands include GRANT and REVOKE.
- **Data query language (DQL)** – Primarily, DQL commands are used to query the data. Most used commands include SELECT which are used to retrieve data from a table.
- **Transaction Control Language** commands are used to change the state of some data -- for example, to COMMIT transaction changes or to ROLLBACK transaction changes.

- As a programming language, SQL has commands and a syntax for issuing those commands.
- SQL was the first commercial language introduced for E.F Codd's **Relational** model of database.
- Today almost all RDBMS(MySql, Oracle, Infomix, Sybase, MS Access etc) use **SQL** as the standard database query language.
- SQL is used to perform all types of data operations in RDBMS.

Creating a Database

- To create a database in RDBMS, **create** command is used.
- Following is the syntax

CREATE DATABASE <DB_NAME>;

- Example CREATE DATABASE Test;
- The above command will create a database named **Test**, which will be an empty schema without any table.

Creating a Table

- Create command can also be used to create tables.
- Now when we create a table, we have to specify the details of the columns of the tables too.
- We can specify the **names** and **datatypes** of various columns in the create command itself.
- Following is the syntax

```
CREATE TABLE <TABLE_NAME>
(
column_name1 datatype1,
column_name2 datatype2,
column_name3 datatype3,
column_name4 datatype4
);
```

- Create table command will tell the database system to create a new table with the given table name and column information.

- Example

```
CREATE TABLE Student(
student_id INT,
name VARCHAR(100),
age INT);
```

- The above command will create a new table with name **Student** in the current database with 3 columns, namely student_id, name and age.
- Where the column student_id will only store integer, name will hold upto 100 characters and age will again store only integer value.
- If you are currently not logged into your database in which you want to create the table then you can also add the database name along with table name, using a dot operator .

```
CREATE TABLE Test.Student(
student_id INT,
name VARCHAR(100),
age INT);
```

- Most commonly used datatypes for Table columns

Datatype	Use
INT	used for columns which will store integer values.
FLOAT	used for columns which will store float values.
DOUBLE	used for columns which will store float values.
VARCHAR	used for columns which will be used to store characters and integers, basically a string.
CHAR	used for columns which will store char values(single character).
DATE	used for columns which will store date values.
TEXT	used for columns which will store text which is generally long in length. For example, if you create a table for storing profile information of a social networking website, then for **about me**section you can have a column of type TEXT.

ALTER command

- Alter command is used for altering the table structure, such as:

  to add a column to existing table

  to rename any existing column

  to change datatype of any column or to modify its size.

  to drop a column from the table.

```
ALTER TABLE table_name ADD(
    column_name datatype);

ALTER TABLE student ADD(
    address VARCHAR(200)
);
```

  - The above command will add a new column address to the table **student**, which will hold data of type varchar which is nothing but string, of length 200.

```
ALTER TABLE table_name ADD(
column_name1 datatype1,
column-name2 datatype2,
 column-name3 datatype3);


ALTER TABLE student ADD(
father_name VARCHAR(60),
mother_name VARCHAR(60),
dob DATE);
```

- The above command will add three new columns to the **student** table
- ALTER command can also be used to modify data type of any existing column. Following is the syntax

```
ALTER TABLE table_name modify(
column_name datatype
);
ALTER TABLE student MODIFY(
address varchar(300));
```

- The above command will modify the address column of the **student** table, to now hold upto 300 characters.

- Using ALTER command can rename an existing column. Following is the syntax

```
ALTER TABLE table_name RENAME
old_column_name TO new_column_name;
```

```
ALTER TABLE student RENAME
address TO location;
```

- The above command will rename address column to location.
- ALTER command can also be used to drop or remove columns. Following is the syntax

```
ALTER TABLE table_name DROP(
column_name);
```

```
ALTER TABLE student DROP(
address);
```

TRUNCATE command

- TRUNCATE command removes all the records from a table.
- But this command will not destroy the table's structure.

```
TRUNCATE TABLE table_name

TRUNCATE TABLE student;
```

- The above query will delete all the records from the table **student**.

DROP command

- DROP command completely removes a table from the database.
- This command will also destroy the table structure and the data stored in it. Following is its syntax

```
DROP TABLE table_name

DROP TABLE student;
```

- The above query will delete the **Student** table completely.
- It can also be used on Databases, to delete the complete database.

```
DROP DATABASE Test;
```

RENAME query

- RENAME command is used to set a new name for any existing table. Following is the syntax

```
RENAME TABLE old_table_name to new_table_name

RENAME TABLE student to students_info;
```

- The above query will rename the table **student** to **students_info**.

INSERT command

- Insert command is used to insert data into a table. Following is its general syntax

```
INSERT INTO table_name VALUES(data1, data2, ...)
```

- Consider a table **student** with the following fields:

s_id	name	age

```
INSERT INTO student VALUES(101, 'Adam', 15);
```

- The above command will insert a new record into **student** table.

s_id	name	age
101	Adam	15

- We can use the INSERT command to insert values for only some specific columns of a row.
- We can specify the column names along with the values to be inserted like this

```
INSERT INTO student(id, name) values(102, 'Alex');
```

- The above SQL query will only insert id and name values in the newly inserted record.

UPDATE command

- UPDATE command is used to update any record of data in a table. Following is its general syntax

```
UPDATE table_name SET column_name = new_value WHERE s
ome_condition;
```

- WHERE is used to add a condition to any SQL query

UPDATE student SET age=18 WHERE student_id=102;

- In the above statement, if we do not use the WHERE clause, then our update query will update age for all the columns of the table to **18**.
- We can also update values of multiple columns using a single UPDATE statement.

```
UPDATE student SET name='Abhi', age=17 where s_id=103;
```

- The above command will update two columns of the record which has s_id 103.

DELETE command

- DELETE command is used to delete data from a table.
- Following is its general syntax

```
DELETE FROM table_name;

DELETE FROM student;
```

- The above command will delete all the records from the table **student**.

- In our **student** table if we want to delete a single record, we can use the WHEREclause to provide a condition in our DELETE statement.

```
DELETE FROM student WHERE s_id=103;
```

- The above command will delete the record where s_id is 103 from the table **student**.

COMMIT command

- COMMIT command is used to permanently save any transaction into the database.
- When we use any DML command like INSERT, UPDATE or DELETE, the changes made by these commands are not permanent, until the current session is closed, the changes made by these commands can be rolled back.
- To avoid that, we use the COMMIT command to mark the changes as permanent
- Following is commit command's syntax

```
COMMIT;
```

ROLLBACK command

- This command restores the database to last commited state.
- It is also used with SAVEPOINT command to jump to a savepoint in an ongoing transaction.
- If we have used the UPDATE command to make some changes into the database, and realise that those changes were not required, then we can use the ROLLBACKcommand to rollback those changes, if they were not commited using the COMMITcommand.
- Following is rollback command's syntax

```
ROLLBACK TO savepoint_name;
```

SAVEPOINT command

- SAVEPOINT command is used to temporarily save a transaction so that we can rollback to that point whenever required.

- Following is savepoint command's syntax

```
SAVEPOINT savepoint_name;

INSERT INTO class VALUES(5, 'Rahul');

COMMIT;

UPDATE class SET name = 'Abhijit' WHERE id = '5’;

SAVEPOINT A;

INSERT INTO class VALUES(6, 'Chris');

SAVEPOINT B;

INSERT INTO class VALUES(7, 'Bravo');

SAVEPOINT C;

SELECT * FROM class;

ROLLBACK TO B;
```

GRANT Command

- Used to provide any user access privileges or other priviliges for the database.

REVOKE Command

- Used to take back permissions from any user

```
GRANT CREATE SESSION TO username;

GRANT CREATE TABLE TO username;

GRANT sysdba TO username

GRANT CREATE ANY TABLE TO username

GRANT DROP ANY TABLE TO username

REVOKE CREATE TABLE FROM username
```

SELECT SQL Query

- SELECT query is used to retrieve data from a table.
- It is the most used SQL query.
- We can retrieve complete table data, or partial by specifying conditions using the WHERE clause.

```
SELECT
column_name1,
column_name2,
column_name3,
...
column_nameN
FROM table_name;
```

```
SELECT s_id, name, age FROM student;
```

- The above query will fetch information of s_id, name and age columns of the **student** table and display them.

```
SELECT * FROM student;
```

- The above query will show all the records of **student** table, that means it will show complete dataset of the table.

```
SELECT * FROM student WHERE name = 'Abhi';
```

Using the WHERE SQL clause

- WHERE clause is used to specify/apply any condition while retrieving, updating or deleting data from a table.
- This clause is used mostly with SELECT, UPDATE and DELETEquery.
- When we specify a condition using the WHERE clause then the query executes only for those records for which the condition specified by the WHERE clause is true.

```
DELETE FROM table_name WHERE [condition];
```

- The WHERE clause is used at the end of any SQL query, to specify a condition for execution.

```
SELECT s_id, name, age, address FROM student WHERE s_id =    101;
```

SELECT s_id, name, age, address FROM student WHERE name = 'Adam';

- Following is a list of operators that can be used while specifying the WHERE clause condition.

Operator	Description
=	Equal to
!=	Not Equal to
<	Less than
>	Greater than
<=	Less than or Equal to
>=	Greate than or Equal to
BETWEEN	Between a specified range of values
LIKE	This is used to search for a pattern in value.
IN	In a given set of values

SQL LIKE clause

- LIKE clause is used in the condition in SQL query with the WHERE clause.
- LIKEclause compares data with an expression using wildcard operators to match pattern given in the condition.
- There are two wildcard operators that are used in LIKE clause.
- **Percent sign %**: represents zero, one or more than one character.
- **Underscore sign** _: represents only a single character.

```
SELECT * FROM Student WHERE s_name LIKE 'A%’;
```

- The above query will return all records where **s_name** starts with character 'A’.

```
SELECT * FROM Student WHERE s_name LIKE '_d%’;
```

- The above query will return all records from **Student** table where **s_name** contain 'd' as second character.

```
SELECT * FROM Student WHERE s_name LIKE '%x';
```

- The above query will return all records from **Student** table where **s_name** contain 'x' as last character.

SQL ORDER BY Clause

- Order by clause is used with SELECT statement for arranging retrieved data in sorted order.
- The **Order by** clause by default sorts the retrieved data in ascending order.
- To sort the data in descending order DESC keyword is used with Order by clause.

```
SELECT column-list|* FROM table-name ORDER BY ASC | DESC;

SELECT * FROM Emp ORDER BY salary;
```

- The above query will return the resultant data in ascending order of the **salary**.

```
SELECT * FROM Emp ORDER BY salary DESC;
```

- The above query will return the resultant data in descending order of the **salary**.

SQL Group By Clause

- Group by clause is used to group the results of a SELECT query based on one or more columns.
- It is also used with SQL functions to group the result from one or more tables.

```
SELECT column_name, function(column_name)
FROM table_name
WHERE condition
GROUP BY column_name
```

```
SELECT name, age
FROM Emp GROUP BY salary
```

```
SELECT name, salary
FROM Emp
WHERE age > 25
GROUP BY salary
```

- Remember that Group By clause will always come at the end of the SQL query, just like the Order by clause.

SQL HAVING Clause

- **Having** clause is used with SQL Queries to give more precise condition for a statement.
- It is used to mention condition in Group by based SQL queries, just like WHERE clause is used with SELECT query.

```
SELECT column_name, function(column_name)
FROM table_name
WHERE column_name condition
GROUP BY column_name
HAVING function(column_name) condition
```

```
SELECT *
FROM sale GROUP BY customer
HAVING sum(previous_balance) > 3000
```

- The main objective of the above SQL query iss to find out the name of the customer who has had a **previous_balance** more than **3000**, based on all the previous sales made to the customer.

DISTINCT keyword

- The distinct keyword is used with SELECT statement to retrieve unique values from the table.
- Distinct removes all the duplicate records while retrieving records from any table in the database.

```
SELECT DISTINCT column-name FROM table-name;

SELECT DISTINCT salary FROM Emp;
```

- The above query will return only the unique salary from **Emp** table.

SQL AND & OR operator

- The AND and OR operators are used with the WHERE clause to make more precise conditions for fetching data from database by combining more than one condition

- AND operator is used to set multiple conditions with the WHERE clause, alongside, SELECT, UPDATE or DELETE SQL queries.

SELECT * FROM Emp WHERE salary < 10000 **AND** age > 25

- The above query will return records where **salary** is less than **10000** and **age**greater than **25**.
- OR operator is also used to combine multiple conditions with WHERE clause.
- The only difference between AND and OR is their behaviour.
- When we use AND to combine two or more than two conditions, records satisfying all the specified conditions will be there in the result.
- But in case of OR operator, atleast one condition from the conditions specified must be satisfied by any record to be in the resultset.

```
SELECT * FROM Emp WHERE salary > 10000 OR age > 25
```

- The above query will return records where **either** salary is greater than 10000 **or**age is greater than 25.

SQL Constraints

- SQL Constraints are rules used to limit the type of data that can go into a table, to maintain the accuracy and integrity of the data inside table.
- Constraints can be divided into the following two types:
- **Column level constraints:** Limits only column data.
- **Table level constraints:** Limits whole table data.
- Constraints are used to make sure that the integrity of data is maintained in the database.

- Following are the most used constraints that can be applied to a table.

    NOT NULL

    UNIQUE

    PRIMARY KEY

    FOREIGN KEY

    CHECK

    DEFAULT

NOT NULL Constraint

- By default, a column can hold NULL values.
- If you do not want a column to have a NULL value, use the NOT NULL constraint.
- It restricts a column from having a NULL value.
-

```
CREATE TABLE Student
(  s_id int NOT NULL,
name varchar(60),
age  int
);
```

- The above query will declare that the **s_id** field of **Student** table will not take NULL value.

UNIQUE Constraint

- It ensures that a column will only have unique values.
- A UNIQUE constraint field cannot have any duplicate data.
- It prevents two records from having identical values in a column

```
CREATE TABLE Student
( s_id int  NOT NULL,
name varchar(60),
age int  NOT NULL UNIQUE
);
```

- The above query will declare that the **s_id** field of **Student** table will only have unique values and wont take NULL value.

Primary Key Constraint

- Primary key constraint uniquely identifies each record in a database.
- A Primary Key must contain unique value and it must not contain null value.
- Usually Primary Key is used to index the data inside the table.

```
CREATE table Student
(s_id int PRIMARY KEY,
Name varchar(60) NOT NULL,
Age int);
```

- The above command will creates a PRIMARY KEY on the s_id.

```
ALTER table Student
ADD PRIMARY KEY (s_id);
```

- The above command will creates a PRIMARY KEY on the s_id.

Foreign Key Constraint

- Foreign key is used to relate two tables.
- The relationship between the two tables matches the Primary Key in one of the tables with a Foreign Key in the second table.
- This is also called a referencing key.

```
CREATE table Order_Detail(
order_id int PRIMARY KEY,
order_name varchar(60) NOT NULL,
c_id int FOREIGN KEY REFERENCES Customer_Detail(c_id)
);
```

- In this query, **c_id** in table Order_Detail is made as foriegn key, which is a reference of **c_id** column in Customer_Detail table.

SQL JOIN

- SQL Join is used to fetch data from two or more tables, which is joined to appear as single set of data.
- It is used for combining column from two or more tables by using values common to both tables.
- JOIN Keyword is used in SQL queries for joining two or more tables.

Types of JOIN

- Following are the types of JOIN that we can use in SQL:
- Inner
- Outer
- Left
- Right
-

INNER Join or EQUI Join

- This is a simple JOIN in which the result is based on matched data as per the equality condition specified in the SQL query.

```
SELECT column-name-list FROM

table-name1 INNER JOIN table-name2

WHERE table-name1.column-name = table-name2.column-name;
```

```
SELECT * from class INNER JOIN class_info where class.id = class_info.id;
```

Natural JOIN

- Natural Join is a type of Inner join which is based on column having same name and same datatype present in both the tables to be joined.

```
SELECT * FROM

table-name1 NATURAL JOIN table-name2;


SELECT * from class NATURAL JOIN class_info;
```

OUTER JOIN

- Outer Join is based on both matched and unmatched data.
- Outer Joins subdivide further into: Left Outer Join, Right Outer Join, Full Outer Join
- The left outer join returns a resultset table with the **matched data** from the two tables and then the remaining rows of the **left** table and null from the **right** table's columns.

SELECT column-name-list FROM

table-name1 LEFT OUTER JOIN table-name2

ON table-name1.column-name = table-name2.column-name;

- To specify a condition, we use the ON keyword with Outer Join.

SELECT column-name-list FROM

table-name1, table-name2 on table-name1.column-name = table-name2.column-name(+);

SELECT * FROM class LEFT OUTER JOIN class_info ON (class.id = class_info.id);

- The right outer join returns a resultset table with the **matched data** from the two tables being joined, then the remaining rows of the **right** table and null for the remaining **left** table's columns.

```
SELECT column-name-list FROM
table-name1 RIGHT OUTER JOIN table-name2
ON table-name1.column-name = table-name2.column-name;
```

```
SELECT * FROM class RIGHT OUTER JOIN class_info ON (class.id =  class_info.id);
```

- The full outer join returns a resultset table with the **matched data** of two table then remaining rows of both **left** table and then the **right** table.

```
SELECT column-name-list FROM
table-name1 FULL OUTER JOIN table-name2
ON table-name1.column-name = table-name2.column-name;
```

```
SELECT * FROM class FULL OUTER JOIN class_info ON (class.id =    class_info.id);
```